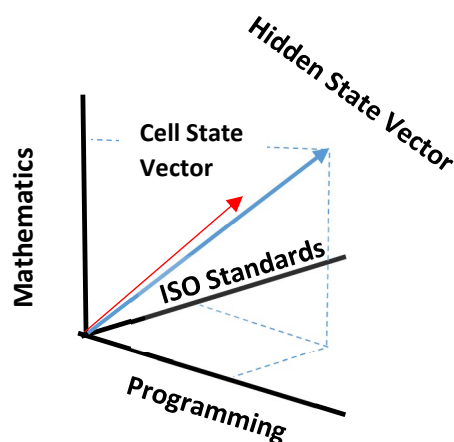


Understanding LSTM ... an intuitive approach

Imagine you registered in a course in latest technologies such as “Information Security”. Being one of the hottest areas in terms of technology evolution, job opportunities, potential growth, you decide to take admission into this course. You gather as much information relating to Information Security as you can to get an idea on what this course will be about. Though you have the entire course structure still the topics being new, you form a mental picture of what the course is likely to be and what you are likely to achieve. For example, you imagine yourself to become a cyber security professional with a vague idea on what exactly will that mean in terms of your role responsibility.

This mental picture about the course in your mind, is your starting point, your initial internal mental state, the hidden state. While attending the first session with your faculty you realize that some of the notions you associated with the course for example a belief that it will be coding intensive is not so and you need to focus more on networking protocols and cryptography. The newly acquired knowledge helps get a better idea of the course and you have an updated internal mental state, the hidden state. This internal state representing your current level of familiarity with information security will undergo further evolution as you cover more and more portions of the course. This is a common learning experience we all go thru. We unlearn some things from the past, internalize new learning and that puts us in a different state than where we were before learning the new thing.

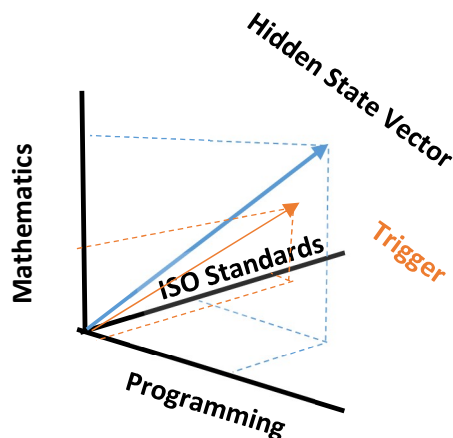
Let us present this learning process in a mathematical way using vectors and vector space as shown below. The vector space represents a state machine. The state variables act as dimensions of the vector space. The internal state of the machine is represented by a vector called “Cell State” (CSV, red vector). This vector is updated when system transitions from one state to next to reflect the current system state. Cell State is updated using another vector called Hidden State Vector (HSV) shown in blue.



1. CS and HSV represent the internal state before attending the first session of the course on Information Security.
2. At every time step HSV is used to update CSV
3. HSV quantifies contribution of each topic (dimension) to the overall course

To understand these multi-dimensional concepts in 3D is very challenging. We should always remember that all these concepts will make sense only when we imagine them playing out in very high dimensional vector space.

Let us assume the expected contribution of each topic to the vector is as shown. Expecting relatively more of maths and ISO standards with some programming. However, in the first class /session, the focus was more on ISO standards, and this led us to believe that the course will demand more in terms of ISO standards and less others (This new information gathered is a trigger). Given the trigger (orange arrow, which has more of ISO compared to Programming and maths), the HSV (blue arrow) must change to reflect this reality in form of the current hidden state.



The contribution of HSV components (Programming, ISO Standards, Mathematics) must change given the trigger. For that we find the relevance of each topic given the trigger (first class / session with focus on ISO)

Relevance of HSV elements in the context of the trigger, can be quantified as the likelihood of programming or mathematical operation given the trigger. For this, we can do elementwise sigmoid on the summation of elements of HSV and Trigger. Whatever the input values, output of Sigmoid operation remains between 0 and 1 and in a way, we are giving different weights to different elements of the HSV given the trigger.

Magnitude change for each element of HSV = $\text{Sigmoid}(\text{Programming}_{\text{hsv}} + \text{Programming}_{\text{trigger}})$, $\text{Sigmoid}(\text{ISO Standards}_{\text{hsv}} + \text{ISO Standards}_{\text{trigger}})$, $\text{Sigmoid}(\text{Maths}_{\text{hsv}} + \text{Maths}_{\text{trigger}})$. This step, in a way assigns importance / weightage to each element of HSV. From the diagram above the weightage assigned to ISO Standards will be larger than programming and mathematics elements.

In LSTM, the HSV represents another vector called the “Cell State Vector” CSV shown as (C_t) in the diagram below. **The CSV represents long term memory while the HSV represents short term memory.** The long term memory is updated at every time step using the short term memory. This is done in two stages where the first step is “Forget Gate”. The purpose of this gate is to fix the relevance of the individual dimensions in the context of the trigger.

Suppose the elementwise Sigmoid function returns following values [0.05, 0.9, 0.15]. **Note, each Sigmoid is independent so their output need not total to 1. They are not to**

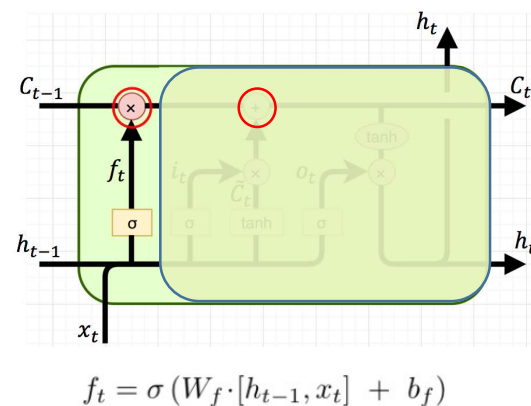
be considered as probability values. These values will be used to modify the CSV through elementwise multiplication (process of forgetting).

Post the modification, the CSV (C_t) becomes a work in progress vector (C_{WIP}). This is the value of CSV between the first multiplication and addition step shown in red circle in the diagram below.

Forget operation

$$C_{WIP} = (C_{\text{Programming}} * 0.05), (C_{\text{ISO}} * 0.9), (C_{\text{Maths}} * 0.15).$$

The Forget Gate is responsible only for creating the C_{WIP} (Cell State Work In Progress).



Forget Gate

1. C_{t-1} is CSV from previous time step (if any)
2. h_{t-1} is HSV from previous time step (if any)
3. x_t is the trigger or input at current time step
4. b is the elementwise sigmoid operation on the HSV and input trigger elements giving f_t
5. $C_{t-1} * f_t$ elementwise operation results in $C_{w.i.p}$
6. b_f is the bias term associated with the linear equation

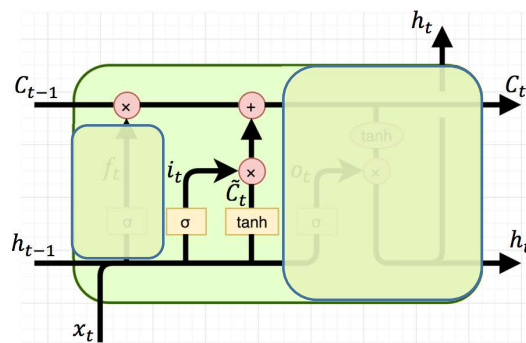
Input Gate

In the next step we must add new information gained in the context of the trigger and use it to update CSV so that it represents the updated Cell state. This is done using the “Input Gate” and a tanh operation.

The same equation based on Sigmoid operation is used on the same two inputs HSV and trigger x but the weights associated with the equation are different from those use in Forget Gate. It is executed again for the same purpose of extracting relevancy of the elements given the trigger x but this time it is done along with another step of extracting mutual information from the elements of the trigger x and the relevant parts of the HSV. This is done using Tanh.

Tanh for Correlation - The output of Tanh ranges between -1 and 1 for all input values and this is the behaviour of correlation function between two entities. Correlation is a standardized representation of covariance between two entities and covariance is a measure of information conveyed by two variables. If you have an attribute A in given data and the value of A is constant across all instances/records, does that attribute carry any information? No. **Constant values do not carry any information about the source / process that generated the data.** On the other hand, variable B has values spread across a central point with some standard deviation. This variable carries behavioural information of the process (in terms of central values and spread) that generated this data. Thus, we say, **variance carries information. So does covariance. Covariance not only carries information about the process but also how one variable interacts with the other.** Covariance expressed in standard deviations is correlation.

In our example, we take the correlation between the trigger X and the HSV elementwise through the Tanh function to quantify how much the two vectors correlate with one another elementwise. The closer Tanh output is to 1 the more is the correlation between the trigger X element and the HSV element. This quantifies the information conveyed by the trigger in the context of the HSV elementwise. How much of this information to carry forward is given by the output of the sigmoid step in the “Input Gate”.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

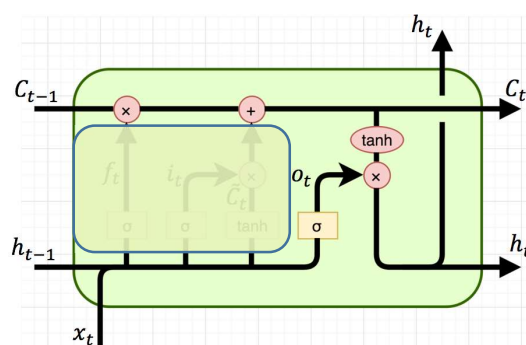
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Input Gate

1. X_t is the trigger or input at current time step
2. σ is the elementwise sigmoid operation on the HSV and input trigger elements giving f_t
3. Elementwise $\tanh(h_{t-1}, X_t)$ extracts mutual information that trigger and HSV convey
4. Sigmoid * Tanh decide the quantum of information carry forward elementwise to new cell state (CSV)
5. The quantum of information is used to update the CSV from $C_{w.i.p}$ to C_t through simple vector addition

Since updating $C_{w.i.p}$ to C_t is done elementwise thru simple addition, the magnitudes of the CSV vector (**C_t**) **elements can become very large**. This is not a problem. However, after updating $C_{w.i.p}$ to C_t , the next step is to use C_t to update HSV (**Output Gate**). For this yet again the Sigmoid output is multiplied with scaled updated C_t element values. The purpose of Tanh function here is to scale the elements of C_t between 1 and -1. This prevents HSV element magnitude from becoming large and this mitigates risk of vanishing gradient problems during training in the subsequent time steps!



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Output Gate

1. Update C_t is used to update HSV h_t
2. Elements of C_t can be large so scale them using tanh
3. Use Sigmoid weighted elements and multiply with tanh of scaled updated C_t to get updated HSV h_t

At every time step one can generate output from h_t . That is what the line going vertically up from update h_t represents. **On completion of the processing at a time step, the LSTM cell outputs both the long-term memory and short term memory to be consumed by next step if any.**

The Sigmoid and Tanh processing is done by layers of neurons respectively. The neurons are associated with weights. Thus there will be weight matrices associated with the “Forget Gate”, “Input Gate” and the “Output Gate”. All these weight matrices help form the gradient vector space in the multidimensional vector space. In this vector field the states of the system are represented by the state attractors and the transition from one state to next is represented by the vector field trajectories.

The weights are learnt during the training process (`model.fit(X_train, y_train)`) using the same backpropagation of error gradients. However, **in LSTM chain (unrolled in time), the backpropagation happens through two channels. One is the path shown as C_t in the pic below and the other through h_t .** The structure of the resulting chain equation (which contains lot additive terms) helps mitigate the risk of vanishing gradient. Refer: <https://medium.datadriveninvestor.com/how-do-lstm-networks-solve-the-problem-of-vanishing-gradients-a6784971a577>

LSTM is considered as old technology now, especially in the context of contemporary advanced Transformer and BERT technologies etc. However, a conceptual understanding of the RNN and LSTM, visualizing their functioning in the virtual vector space as stochastic state machines will help understand the latest technologies.

Summary :

1. To understand sequential models starting with RNN and its variants to the latest ones such as Transformers, understanding the word vectorization (in contrast to document vectorization) is a must.
2. In word vectorization understanding Word2Vec (CBOW, Skip Gram), Glove and other methodologies are important. Understanding the vector space in terms of its dimensions i.e., what the various dimensions represent is important.
 - a. The dimensions are like latent topics that we come across topics identification using matrix factorization techniques such as LDA. The latent topics are hidden features (that is why they are latent) of the corpus of documents.
 - b. Vectorizing documents in a feature space of those latent dimensions helps define clusters of documents where a cluster signifies some sort of a context which characterize the documents
 - c. Similarly, in word vectorization, the dimensions are akin to latent topics in the context of which the words are used to convey some meaning/ information.
3. Some words may mean different things depending on the context in which they are being used. Such as “Train” which could be referring to a locomotive or teaching and only the surrounding words will help us address this ambiguity.
4. The context of a word is formed by the cluster of words of which it is a member and clusters will be good clusters only when the number of dimensions (latent features) are right. As in topic identification, a-priori knowledge of how many latent features is there in the corpus is a question with no answer. We must try different values to find

the most likely number of latent features. Embedding words in a vector space with the right number of latent features helps build an effective model

5. When we learn something new, our own leaning happens in stages. We carry some pre-fixed ideas and on getting exposed to the new concept, dilute some of the ideas we came with and build on some. This is what the “Forget Gate” and “Input Gate” embody.
6. To retain the new knowledge, we need to practice what we learnt. This transfers the learnings from short term memory to long term memory. Same functionality is achieved by the process of using HSV to update CSV and finally use CSV to update HSV

References:

1. Modelling stochastic correlation – Long Teng, Matthias Ehrhardt, Michael Gunther
2. Information Extraction / Speech and Language Processing – Daniel Jurafsky, James H Martin
3. Short Term Memory – Emanuel Guigon, Yves Burnod
4. Arithmetic properties of word embeddings – Hicham EL BOUKKOURI
5. Computing With Attractors – John Hertz
6. Identification and Control – Kumpati. S. Narendra
7. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> - Christopher Olah
8. <https://medium.datadriveninvestor.com/how-do-lstm-networks-solve-the-problem-of-vanishing-gradients-a6784971a577> - Nir Arbel